

Trees and Recursive Datatypes

CS 350

Dr. Joseph Eremondi

January 21, 2025

Recursive Algebraic Data Types

Recursive Algebraic Data Types

Self-reference in types

- Datatypes are made more powerful when we allow self reference

Self-reference in types

- Datatypes are made more powerful when we allow self reference
- This allows us to define **trees**

Self-reference in types

- Datatypes are made more powerful when we allow self reference
- This allows us to define **trees**
 - of arbitrary depth

Self-reference in types

- Datatypes are made more powerful when we allow self reference
- This allows us to define **trees**
 - of arbitrary depth
- Data can be traversed using recursion

- In other classes, you might have seen specific kinds of trees

- In other classes, you might have seen specific kinds of trees
 - Binary search trees

- In other classes, you might have seen specific kinds of trees
 - Binary search trees
 - 2-3 Trees, BTrees, Red-Black trees

Trees vs. Trees™

- In other classes, you might have seen specific kinds of trees
 - Binary search trees
 - 2-3 Trees, BTrees, Red-Black trees
- In this class, we will mean tree very generally

- In other classes, you might have seen specific kinds of trees
 - Binary search trees
 - 2-3 Trees, BTrees, Red-Black trees
- In this class, we will mean tree very generally
 - Anything that has a hierarchical structure

- In other classes, you might have seen specific kinds of trees
 - Binary search trees
 - 2-3 Trees, BTrees, Red-Black trees
- In this class, we will mean tree very generally
 - Anything that has a hierarchical structure
- Everything in a tree is either a leaf, or has 1 or more children

- In other classes, you might have seen specific kinds of trees
 - Binary search trees
 - 2-3 Trees, BTrees, Red-Black trees
- In this class, we will mean tree very generally
 - Anything that has a hierarchical structure
- Everything in a tree is either a leaf, or has 1 or more children
 - That are also trees

- In other classes, you might have seen specific kinds of trees
 - Binary search trees
 - 2-3 Trees, BTrees, Red-Black trees
- In this class, we will mean tree very generally
 - Anything that has a hierarchical structure
- Everything in a tree is either a leaf, or has 1 or more children
 - That are also trees
- What is in the tree depends on the kind of tree

Example: Lists as a datatype

Example: Lists as a datatype

```
(define-type NumList  
  (Nil)  
  (Cons [head : Number] [tail : NumList]))
```

- Note: this is not quite how lists are defined in Racket/plait

Example: Lists as a datatype

```
(define-type NumList  
  (Nil)  
  (Cons [head : Number] [tail : NumList]))
```

- Note: this is not quite how lists are defined in Racket/plait
 - But they could be!

Example: Lists as a datatype

```
(define-type NumList  
  (Nil)  
  (Cons [head : Number] [tail : NumList]))
```

- Note: this is not quite how lists are defined in Racket/plait
 - But they could be!
- Recursive fields in datatype → recursive calls in template

Example: Lists as a datatype

```
(define-type NumList  
  (Nil)  
  (Cons [head : Number] [tail : NumList]))
```

- Note: this is not quite how lists are defined in Racket/plait
 - But they could be!
- Recursive fields in datatype → recursive calls in template

Example: Lists as a datatype

```
(define-type NumList
  (Nil)
  (Cons [head : Number] [tail : NumList]))
```

- Note: this is not quite how lists are defined in Racket/plait
 - But they could be!
- Recursive fields in datatype → recursive calls in template

```
(define (sum [xs : NumList])
  (type-case NumList xs
    [(Nil)
     0]
    [(Cons h t)
     (let ([sumRest (sum t)])
       (+ h (sumRest)))]))
(sum (Cons 100 (Cons 20 (Cons 3 (Nil)))))
```

Lists are Trees

- A list is a tree where:

Lists are Trees

- A list is a tree where:
 - Each node has 0 or 1 children (linear)

Lists are Trees

- A list is a tree where:
 - Each node has 0 or 1 children (linear)
 - 1-child nodes contain data: head of list

Lists are Trees

- A list is a tree where:
 - Each node has 0 or 1 children (linear)
 - 1-child nodes contain data: head of list
- Similarly, a natural number is a tree where

Lists are Trees

- A list is a tree where:
 - Each node has 0 or 1 children (linear)
 - 1-child nodes contain data: head of list
- Similarly, a natural number is a tree where
 - Each child has 0 or 1 nodes

Lists are Trees

- A list is a tree where:
 - Each node has 0 or 1 children (linear)
 - 1-child nodes contain data: head of list
- Similarly, a natural number is a tree where
 - Each child has 0 or 1 nodes
 - No data is stored in any nodes

Example: Filesystem

- Model of a file system

Example: Filesystem

- Model of a file system
 - Not how is implemented on disk

Example: Filesystem

- Model of a file system
 - Not how is implemented on disk

Example: Filesystem

- Model of a file system
 - Not how is implemented on disk

```
(define-type Filesystem
  (File [name : String]
        [data : Number])
  (Folder [name : String]
           [contents : (Listof Filesystem)]))
```

Linear-time Search using Recursion

- Find the first matching file

Linear-time Search using Recursion

- Find the first matching file

Linear-time Search using Recursion

- Find the first matching file

```
(define (search [target : String]
               [fs : Filesystem]) : (Optionof Number)
  (type-case Filesystem fs
    [(File name data)
     (if (string=? name target)
          (some data)
          (none))])
    [(Folder _ contents)
     (searchList target contents)]))
```

Helper Function: Searching the list

- We have mutually recursive types

Helper Function: Searching the list

- We have mutually recursive types
 - Filesystem contains (Listof Filesystem)

Helper Function: Searching the list

- We have mutually recursive types
 - `Filesystem` contains `(Listof Filesystem)`
 - `(Listof Filesystem)` contains `Filesystem`

Helper Function: Searching the list

- We have mutually recursive types
 - `Filesystem` contains `(Listof Filesystem)`
 - `(Listof Filesystem)` contains `Filesystem`
- So we use mutually-recursive functions

Helper Function: Searching the list

- We have mutually recursive types
 - `Filesystem` contains `(Listof Filesystem)`
 - `(Listof Filesystem)` contains `Filesystem`
- So we use mutually-recursive functions

Helper Function: Searching the list

- We have mutually recursive types
 - Filesystem contains (Listof Filesystem)
 - (Listof Filesystem) contains Filesystem
- So we use mutually-recursive functions

```
(define (searchList [target : String]
                   [files : (Listof Filesystem)])
  : (Optionof Number)
  (type-case (Listof Filesystem) files
    [empty (none)]
    [(cons h t)
     (let ([result (search target h)])
       (if (none? result)
           (searchList target t)
           result))
     ]))
```


Testing the search

Testing the search

```
(define InnerSolarSystem
  (Folder "Sun"
    (list (File "Mercury" 1)
          (File "Venus" 2)
          (Folder "EarthSystem"
            (list (File "Earth" 3)
                  (File "Moon" 3.5)))
          (Folder "MarsSystem"
            (list (File "Mars" 4)
                  (File "Phobos" 4.3)
                  (File "Deimos" 4.6))))))

(search "Moon" InnerSolarSystem)
(search "Jupiter" InnerSolarSystem)
```

Testing the search

```
(define InnerSolarSystem
  (Folder "Sun"
    (list (File "Mercury" 1)
          (File "Venus" 2)
          (Folder "EarthSystem"
            (list (File "Earth" 3)
                  (File "Moon" 3.5)))
          (Folder "MarsSystem"
            (list (File "Mars" 4)
                  (File "Phobos" 4.3)
                  (File "Demos" 4.6))))))

(search "Moon" InnerSolarSystem)
(search "Jupiter" InnerSolarSystem)
```

```
(some 3.5)
(none)
```

Example: Binary Search Tree

- Recall from CS 210

Example: Binary Search Tree

- Recall from CS 210
- A Binary search tree (BST) stores data indexed by some key

Example: Binary Search Tree

- Recall from CS 210
- A Binary search tree (BST) stores data indexed by some key
- Each node is empty, or has data and 2 (possibly empty) children

Example: Binary Search Tree

- Recall from CS 210
- A Binary search tree (BST) stores data indexed by some key
- Each node is empty, or has data and 2 (possibly empty) children
- Finding a node in the BST is linear *in the depth of the tree*

Example: Binary Search Tree

- Recall from CS 210
- A Binary search tree (BST) stores data indexed by some key
- Each node is empty, or has data and 2 (possibly empty) children
- Finding a node in the BST is linear *in the depth of the tree*
 - $O(\log n)$ time if it's balanced

BST data definition

```
;; A BST is either empty (with no data)  
;; or has a data, a binary search tree with smaller data  
(define-type (BST 'elem)  
  (leaf)  
  (node [left : (BST 'elem)]  
        [right : (BST 'elem)]  
        [index : Number]  
        [data : 'elem]))
```

BST Range Check

- Make sure that a BST is well-formed, and that all its data is in the given range

BST Range Check

- Make sure that a BST is well-formed, and that all its data is in the given range
- Only use this in testing

BST Range Check

- Make sure that a BST is well-formed, and that all its data is in the given range
- Only use this in testing

BST Range Check

- Make sure that a BST is well-formed, and that all its data is in the given range
- Only use this in testing

```
(define (BST-inrange [t : (BST 'elem)]
                  [strict-lower-bound : Number]
                  [upper-bound : Number]) : Boolean
  (type-case (BST 'elem) t
    ;; A tree with no data is always in range
    [(leaf) #t]
    ;; non-empty: check if index in range
    ;; and that left/right are all smaller/bigger
    [(node left right index data)
     (and
      (> index strict-lower-bound)
      (<= index upper-bound)
      ;; Left tree should be in range, and <= this node
      (BST-inrange left strict-lower-bound index)
      ;; Right tree should be in range, and > this node
      (BST-inrange right index upper-bound))]))
```

Well Formed BSTs

- Call our range-checker with default values

Well Formed BSTs

- Call our range-checker with default values

Well Formed BSTs

- Call our range-checker with default values

```
(define (BST-wellformed [t : (BST 'elem)]) : Boolean
  ;; We check if all nodes are in the correct range,
  ;; with a starting range
  ;; of -infinity to +infinity i.e. all numbers
  (BST-inrange t -inf.0 +inf.0))
```

Adding a node

- Make a new tree that contains the data under the given tree

Adding a node

- Make a new tree that contains the data under the given tree
 - Kind of like cons, but for BSTs

Adding a node

- Make a new tree that contains the data under the given tree
 - Kind of like cons, but for BSTs

Adding a node

- Make a new tree that contains the data under the given tree
 - Kind of like cons, but for BSTs

```
(define (BST-insert [index : Number]
                    [data : 'elem]
                    [t : (BST 'elem)]) : (BST 'elem))
  (type-case (BST 'elem) t
    ;; Inserting a node into the empty tree
    ;; results in a 1-node tree
    [(leaf) (node (leaf) (leaf) index data)]
    ;; If the tree isn't empty, then either add to left or right,
    ;; depending if new index is smaller/bigger than this node
    [(node left right nodeIndex nodeData)
     (let* ([newLR
              (if (<= index nodeIndex)
                  (pair (BST-insert index data left) right)
                  (pair left (BST-insert index data right)))]
            (node (fst newLR) (snd newLR) nodeIndex nodeData)))]
```

Looking up a node

- Find the all the data for a given tree

Looking up a node

- Find the all the data for a given tree
 - Might have none or duplicates, so return a list

Looking up a node

- Find the all the data for a given tree
 - Might have none or duplicates, so return a list

Looking up a node

- Find the all the data for a given tree
 - Might have none or duplicates, so return a list

```
(define (BST-lookup [t : (BST 'elem)]
                  [index : Number]) : (Listof 'elem)
  (type-case (BST 'elem) t
    [(leaf) '()]
    [(node left right nodeIndex data)
     ;; Look left or right,
     ;; depending if target is less than this node
     (let*
       ([child-data
        (if (<= index nodeIndex)
            (BST-lookup left index)
            (BST-lookup right index))])
       ;; Finally, we check if this node contains the target,
       ;; and if it does we include it in the results.
       (if (= index nodeIndex)
           (cons data child-data)
           child-data))]))
```

Trying it out

Trying it out

```
(define t (BST-insert 3 "three"  
  (BST-insert 5 "five"  
    (BST-insert 99 "ninety-nine"  
      (BST-insert 0 "zero"  
        (BST-insert 99 "NineNine" (leaf)))))))  
(BST-lookup t 99 )
```

Trying it out

```
(define t (BST-insert 3 "three"  
  (BST-insert 5 "five"  
    (BST-insert 99 "ninety-nine"  
      (BST-insert 0 "zero"  
        (BST-insert 99 "NineNine" (leaf)))))))  
(BST-lookup t 99 )
```

```
'("NineNine" "ninety-nine")
```

A Sneak Preview

- Math expressions are trees

A Sneak Preview

- Math expressions are trees

- Math expressions are trees

```
(define-type Expr
  (NumE)
  (PlusE [x : Expr] [y : Expr])
  (TimesE [x : Expr] [y : Expr]))
```

- We'll learn to evaluate these in the 2nd half of the course